

FUNDAMENTOS DE PROGRAMAÇÃO

DOM



Teach
3035

Índice

DOM.....	03
O que é?.....	03
Estrutura.....	04
Onde ele está inserido?.....	06
Manipulação do DOM.....	06

DOM



O que é?



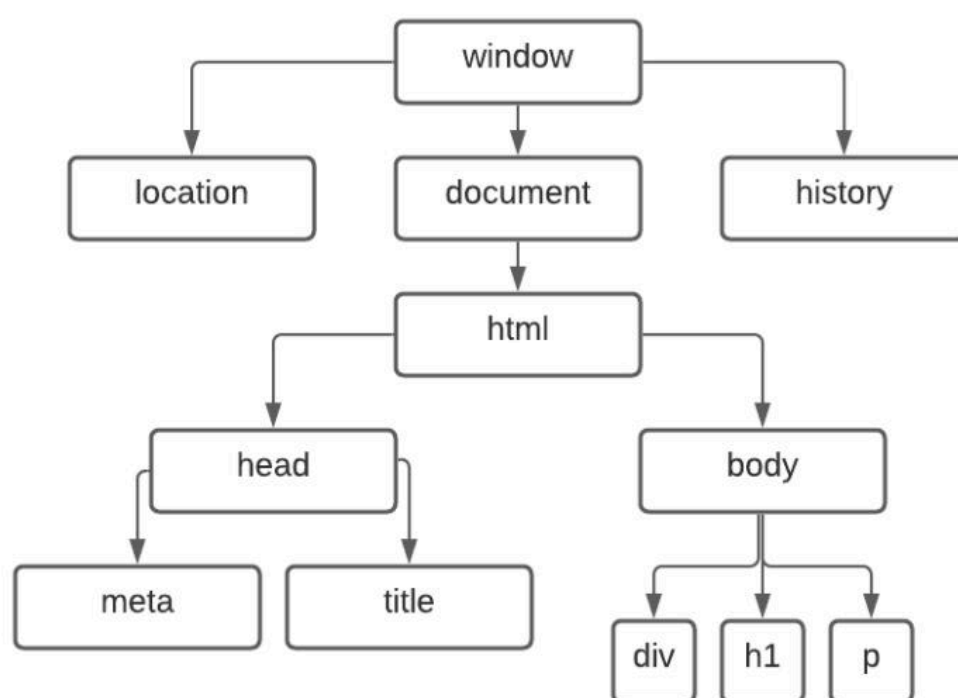
No JavaScript, o termo "dom" refere-se ao **Document Object Model** (Modelo de Objeto de Documento). O DOM é uma representação em forma de árvore de todos os elementos de um documento HTML ou XML que podem ser manipulados usando JavaScript.

O DOM é uma parte fundamental da programação em JavaScript para trabalhar com páginas web interativas e dinâmicas, permitindo que você acesse, altere e manipule os elementos de uma página.

Estrutura



Como já mencionado anteriormente, o DOM é organizado em forma de árvore hierárquica, representando a estrutura dos elementos de um documento HTML ou XML.



Exemplo de uma árvore DOM de uma página Web

No topo dessa árvore, temos o objeto global **window**, que representa a janela do navegador. Em seguida, temos o objeto **document**, que atua como o elemento raiz da estrutura do DOM.

Estrutura



A partir daí, ramificações descendem do **document**, representando as diferentes partes do documento HTML. A tag **<html>** é o objeto pai e contém os objetos filhos **<head>** e **<body>**, correspondendo ao cabeçalho e ao corpo da página, respectivamente.

Essa estrutura hierárquica continua com os objetos filhos dentro do **<head>** e **<body>** do . Por exemplo, o objeto pai **<head>** pode conter o objeto filho **<title>**, que representa o título exibido na barra de título do navegador. Da mesma forma, o objeto pai **<body>** pode conter objetos filhos como as tags **<h1>**, **<div>**, **<p>**.

As tags pai são referidas como "parent" em relação às suas tags filhas, que são chamadas de "child". A estrutura hierárquica mencionada anteriormente, ficaria assim no código HTML:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Título da Página</title>
  </head>
  <body>
    <h1>Título Principal</h1>
    <div>
      <p>Este é um parágrafo de exemplo.</p>
    </div>
  </body>
</html>
```

Onde ele está inserido?



O DOM não está inserido em um arquivo específico. Ele é criado dinamicamente pelo navegador ao interpretar e renderizar o código HTML de um arquivo ou página web. Então, para que ele funcione corretamente, não é necessário realizar nenhuma ação específica, pois ele é uma parte essencial do navegador, basta sua estrutura HTML ser válida.

Manipulação do DOM



Existem diversas maneiras de navegar pelo DOM, e no JavaScript, utilizamos o objeto **document** para acessar as propriedades e métodos disponíveis. Isso nos permite selecionar, modificar, remover e criar elementos nos componentes de um website.

Vamos utilizar esse código HTML como base para explorarmos formas de manipular o DOM:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Aprendendo DOM</title>
  </head>
  <body>
    <h1>Lista de Exemplo</h1>
    <div class="container">
      <ul id="name-list">
        <li>Maria</li>
        <li>João</li>
        <li>Pedro</li>
        <li>Cláudia</li>
      </ul>
    </div>
  </body>
</html>
```

Manipulação do DOM



- **document.documentElement:** Retorna o elemento raiz de uma página HTML.

```
> let html = document.documentElement
< undefined
> html
< <html>
  > <head>...</head>
  > <body>...</body>
  </html>
```

- **document.getElementById():** Retorna o elemento HTML com o ID especificado.

```
> let nameList = document.getElementById('name-list');
< undefined
> nameList
< ▼ <ul id="name-list">
  > <li>...</li>
  > <li>...</li>
  > <li>...</li>
  > <li>...</li>
  </ul>
```

Manipulação do DOM



- **document.getElementsByClassName():** Retorna uma coleção de elementos HTML que possuem a classe especificada.

```
> let container = document.getElementsByClassName('container')
< undefined
> container
< ▶ HTMLCollection [div.container]
> container[0]
< ▶ <div class="container">...</div>
```

- **getElementsByTagName():** Retorna uma coleção de elementos HTML com a tag especificada.

```
> let liTags = document.getElementsByTagName("li")
< undefined
> liTags
< ▼ HTMLCollection(4) [li, li, li, li] ⓘ
  ▶ 0: li
  ▶ 1: li
  ▶ 2: li
  ▶ 3: li
  length: 4
  ▶ [[Prototype]]: HTMLCollection
```


Manipulação do DOM



- o **document.querySelector():** Retorna o primeiro elemento que corresponde a um seletor CSS especificado dentro do documento. Ele permite selecionar elementos com base em classes, IDs, tags e outros seletores CSS

```
> let body = document.querySelector('body')
< undefined
> let list = document.querySelector('#name-list')
< undefined
> body
< ▼<body>
  <h1>Lista de Exemplo</h1>
  ▼<div class="container">
    ▶<ul id="name-list">...</ul>
  </div>
</body>
> list
< ▼<ul id="name-list">
  ▶<li>...</li>
  ▶<li>...</li>
  ▶<li>...</li>
  ▶<li>...</li>
</ul>
```

- o **document.querySelectorAll():** Diferentemente do `querySelector()`, ele não retorna apenas o primeiro elemento correspondente, mas todos os elementos encontrados.

Manipulação do DOM



- **document.createElement():** Cria um novo elemento HTML com o nome da tag especificada, permitindo a criação dinâmica de elementos que podem ser adicionados a uma página.

```
> let newLiElement = document.createElement('li');  
< undefined  
> newLiElement  
< <li></li>
```

- **document.createAttribute():** Cria um novo atributo HTML, permitindo a criação dinâmica de atributos que podem ser adicionados a elementos específicos.

```
> newLiElement.setAttribute('id', 'novoElemento')  
< undefined  
> newLiElement  
< <li id="novoElemento"></li>
```

Manipulação do DOM



- o **element.appendChild():** Insere um novo nó como filho no final de um elemento especificado, permitindo adicionar elementos ou conteúdo adicional à estrutura de um documento HTML.

```
> const nameList = document.getElementById('name-list')
< undefined
> nameList
< ▼ <ul id="name-list">
  ▶ <li>...</li>
  ▶ <li>...</li>
  ▶ <li>...</li>
  ▶ <li>...</li>
  </ul>
> nameList.appendChild(newLiElement)
< ▶ <li id="novoElemento">...</li>
> nameList
< ▼ <ul id="name-list">
  ▶ <li>...</li>
  ▶ <li>...</li>
  ▶ <li>...</li>
  ▶ <li>...</li>
  ▶ <li id="novoElemento">...</li>
  </ul>
```

Manipulação do DOM



- **document.createTextNode():** Cria um novo nó de texto que pode ser inserido em elementos HTML, permitindo adicionar conteúdo de texto apropriado usando JavaScript.

```
> let newText = document.createTextNode('Novo item')
< undefined
> newLiElement.appendChild(newText)
< "Novo item"
> newLiElement
< ▾<li id="novoElemento">
  ::marker
  "Novo item"
</li>
```

Manipulação do DOM



- o **element.removeChild():** Remove um nó filho específico de um elemento, permitindo remover elementos ou conteúdo de um documento HTML.

```
> nameList
< ▼<ul id="name-list">
  ▼<li>
    ::marker
    "Maria"
    </li>
  ▶<li>_</li>
  ▶<li>_</li>
  ▶<li>_</li>
  ▶<li id="novoElemento">_</li>
  </ul>

> let firstItem = nameList.querySelector('li')
< undefined
> firstItem
< ▼<li>
  ::marker
  "Maria"
  </li>

> nameList.removeChild(firstItem)
< <li>Maria</li>
> nameList
< ▼<ul id="name-list">
  ▼<li>
    ::marker
    "João"
    </li>
  ▶<li>_</li>
  ▶<li>_</li>
  ▶<li id="novoElemento">_</li>
  </ul>
```

Manipulação do DOM



- o **element.style.property:** é uma forma de acessar e modificar os estilos inline de um elemento HTML.

```
> var liElement = document.getElementById('name-list').querySelector('li');  
< undefined  
> liElement.style.color = 'red';  
< 'red'
```

O resultado será esse:

Lista de Exemplo

- João
- Pedro
- Cláudia
- Novo item

Aqui foram abordados apenas alguns dos métodos e funcionalidades do DOM, portanto, para obter informações mais abrangentes sobre os métodos e propriedades do DOM é importante consultar a documentação: **[Clique aqui para ver a documentação completa.](#)**